First Steps with Your QNX System

Lecture 2

Topic 0 - What is the QNX CTI (Custom Target Image) project

What is the QNX CTI (Custom Target Image) project?

The CTI Guide provides instructions on how to set up the Custom Target Image on a Raspberry Pi 4, Raspberry Pi 5, or QEMU (x86_64). qnx It is an open-source GitLab project (Apache 2.0) that gives you a fully reproducible, build-from-source SD card image running QNX OS 8.0. Unlike the Quick Start Target Image (QSTI), which you simply flash pre-built, the CTI is designed to be *modified* — you edit source files, run `make`, and produce your own image.

The core build engine is `mkqnximage`, which reads all the project's configuration, assets, and snippet fragments and assembles the final SD card image containing three partitions: boot, system, and data.

QNX Everywhere

QNX

QNX CTI (Custom Target Image) project

Prerequisites

Linux Utilities:

```
$ sudo apt install automake bridge-utils cmake curl g++ git imagemagick
libglib2.0-bin libglib2.0-dev libssl-dev libtool libwayland-bin libzstd-dev
lua-zlib lua-zlib-dev make ncat ninja-build pax-utils pkg-config python3-pip
sassc scdoc texinfo unzip wget

$ pip3 install gi-docgen markdown packaging pygments strenum toml tomli
typogrify
```

QNX Software: A QNX Software Development Platform (version 8.0) license and an installation of the QNX Software Center (QSC) is required to build this project.

QNX CTI (Custom Target Image) project

Getting Started

1- Clone this repo and navigate into the project folder.
2- Export path to the **qnxsoftwarecenter_clt** executable (modify accordingly to your installation location)

```
export QSC_CLT_PATH=$HOME/qnx/qnxsoftwarecenter/qnxsoftwarecenter_clt
```

3- Create a file called "**options_file**" and populate it with:

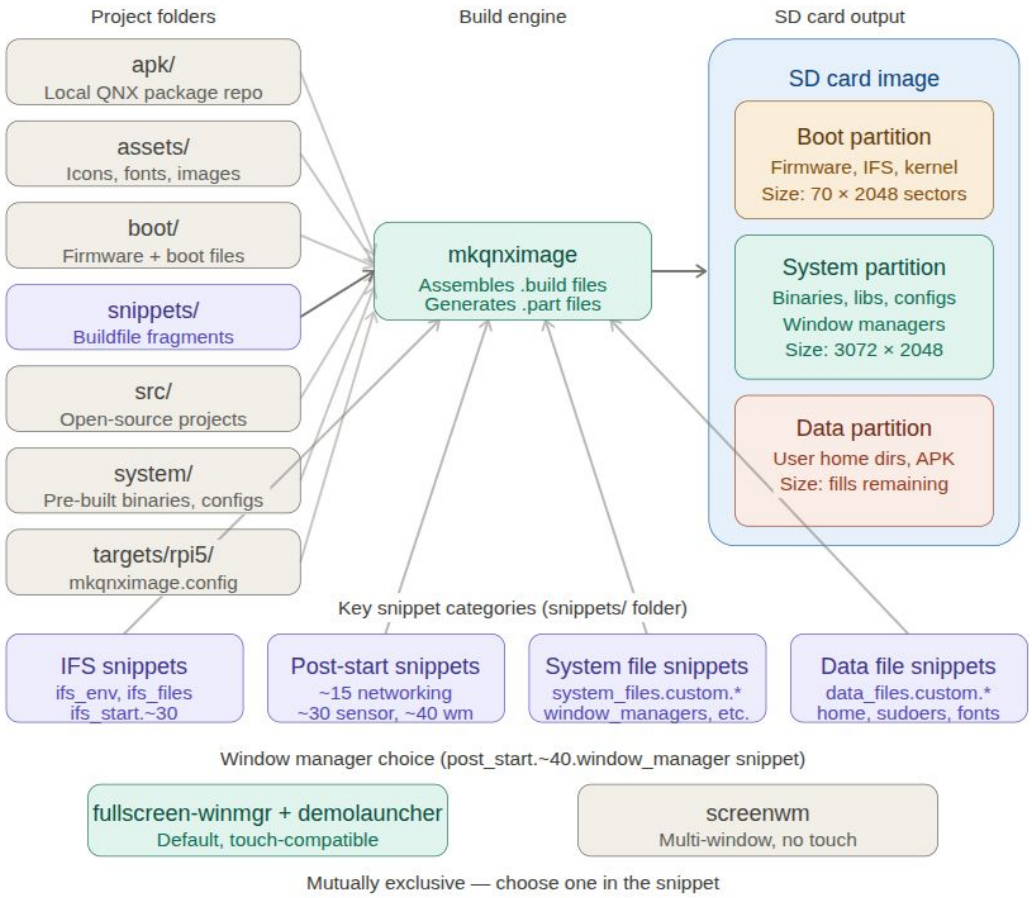
```
-myqnx.user
<username>
-myqnx.password
<password>
```

(Replace the placeholders with your qnx.com credentials)
This file is used to provide your credentials to QNX Software Center (QSC) to install required packages locally for the image build. Take care not to share this file with others or commit it to a fork of this project.
Once the above steps are performed, execute make while specifying your desired target.

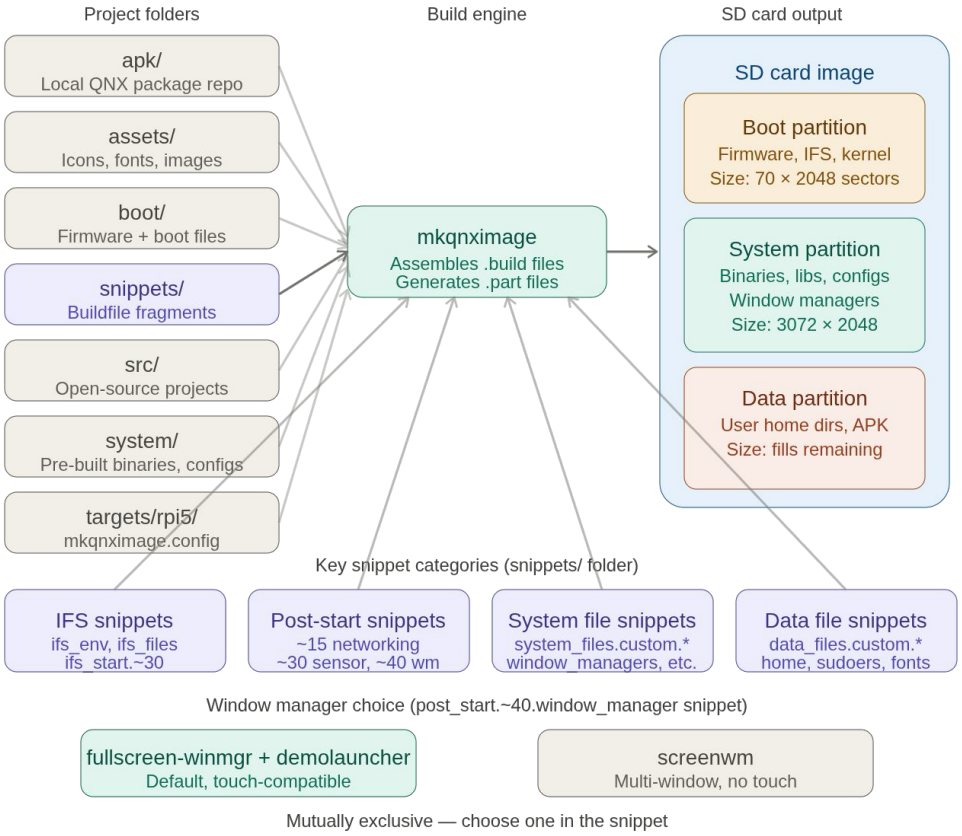
4- Building RPi5 `make TARGET=rpi5`

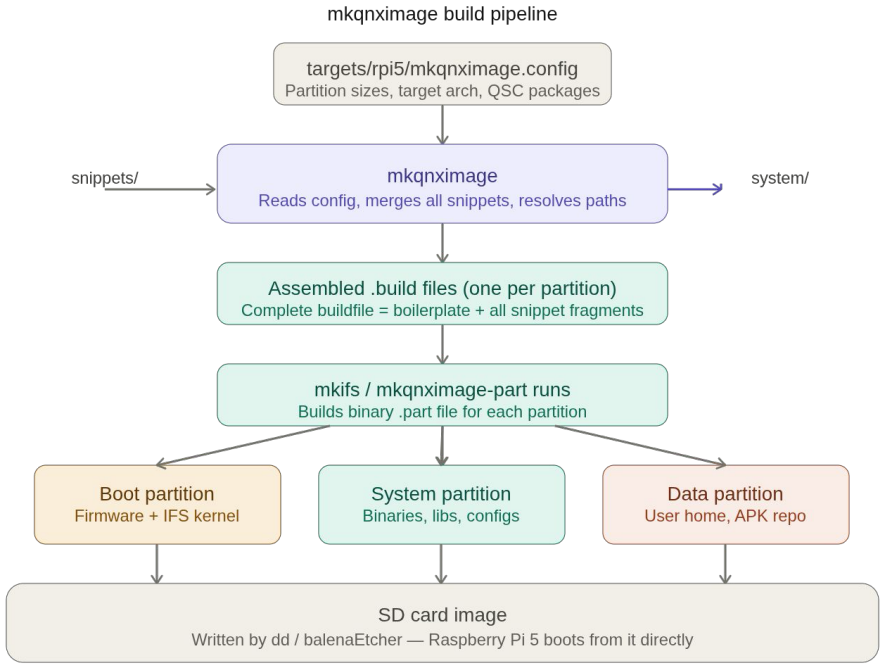
If the build is successful, it will produce the image file **build/rpi5/rpi5.img**.

QNX CTI project structure & build pipeline



QNX CTI project structure & build pipeline





QNX CTI project Details

Configuration reading. `mkqnximage` first reads

- `targets/rpi5/mkqnximage.config`. This file is the entry point — it tells the engine three critical things: what hardware architecture to target (`rpi5`), how big to make each partition (the `OPT_PART_SIZES` triplet), and which QSC packages to install locally into the `apk/` folder before assembly begins.

```
EXTRA_DIRS=+$MKQNXIMAGE_EXTRAS/rpi5:+$MKQNXIMAGE_EXTRAS/cti:+qnx
OPT_TYPE=rpi5
OPT_REPOS='$BOOT:$BOOT/boot:$SYSTEM:$BSP/prebuilt:$BSP/install:$ASSETS/fonts/build/stage:$ASSETS:
OPT_PART_SIZES='800:10240:32000'
OPT_USB=yes
OPT_PYTHON=yes
OPT_GRAPHICS=yes
OPT_SLM=yes
OPT_SYS_INODES=500000
OPT_DATA_INODES=500000
OPT_TIME_SERVERS=$TIME_SERVERS
OPT_ROOT=no
OPT_SECURE_PROCFS=no
OPT_SSH_IDENT=$BUILD/root_authorized_keys
OPT_COPY_DEST=$BUILD/rpi5.img
OPT_HOSTNAME=qnxawadin
```

QNX CTI project Details

Resizing Partitions

`OPT_PART_SIZES='800:10240:32000'`

> As you customize this project with additional assets and open source software, you may need to increase the size of the **system** partition. The **data** partition may also need to be resized if assets are integrated into the data partition for users.

Defines the size of the three partitions in units of 2048-byte sectors. So:

- Boot partition: 800 × 2048 = ~1.6 MB
- System partition: 10240 × 2048 = ~20 MB
- Data partition: 32000 × 2048 = ~62.5 MB (The third size is optional, as the default behavior is to create a data partition that fills the available space.)

This has been increased from the default (which was `70:3072:32000`) — the system partition is now ~20 MB, suggesting a larger set of binaries is being included, possibly additional drivers, open-source builds, or hypervisor components.

QNX CTI project Details

Adding or changing QNX Software Center (QSC) packages

The QNX SOftware Center (QSC) Command Line Tool is used to install a local SDP into **qnx800** that will be used to build the desired target image. The tool is provided an explicit list of packages, and their associated versions, to install. Since this list depends on the target being built, switching between targets may cause the local SDP to be re-installed.

The **qsc_install_packages.list** file contains the list of packages that are installed at the beginning of the build to set up a local QNX SDP installation for the build.

Adding extra packages shouldn't break the build immediately. However, you need to perform extra steps to integrate package contents for additional packages into the image. Refer to the "snippets" and "system" sections in this chapter for more details on which snippets would have to be updated to integrate the content from new packages.

targets/<< target >>/qsc_install_packages.list

This file contains the list of packages that are relevant to the specific target. It is processed after the qsc_install_pacakges.txt file in the root of the repo. If you want to add something to a specific target, say something to support HW specific to the target, this is the file you would want to modify.

QNX CTI project Details

The snippets system

The `snippets` folder contains portions of buildfiles that `mkqnximage` uses to create partitions for the SD card image generated by the project build. The `mkqnximage` utility combines these files with boilerplate snippets to assemble full buildfiles (`.build`), that are required to generate partition files (`.part`) for the image generation process.

The engine concatenates all fragments of the same type into one complete buildfile. This is why snippets are the *primary customization interface* — you never edit the boilerplate buildfiles directly; you insert fragments at the correct numeric position.

Snippet entries use a `destination=source` syntax where the source path is relative to the project root. At this stage `mkqnximage` resolves those paths — finding your binaries in `system/bin/`, your configs in `system/etc/`, your staged open-source builds in `src/stage/`, and your shared assets copied in from `assets/`. If any path cannot be resolved, the build fails here with a clear error before anything is written to disk.

QNX CTI project Details

Combination Rules

The snippets folder contains snippets of build files that are combined with some boilerplate build segments to generate final build files for the three partitions: **(boot - data - system)**

- The initial combination is done by combining the common snippet files with the target snippet files(eg. /targets/rpi5/snippets). If a target snippet file has the same name as a common one, then the target specific one wins and is used; the common one is ignored in this case.
- After this initial combination, mkqnximage has specific rules governing how individual snippets are used. Each name of each snippet file has a **prefix** that determines which build file, script or other file it is included in, a **suffix** to make the file name unique, and optionally a **string of the form 'NN' or '~NN'** that can be used to influence ordering with respect to other files using the same prefix.

```
<prefix>.<suffix>     - Inserted in default postion
<prefix>.80.<suffix> - Inserted earlier relative to others
<prefix>.~80.<suffix>- Inserted after all non-tilde files
                       files that include ~
Note: Smaller number → inserted earlier
      Larger number → inserted later
```

QNX

QNX CTI project Details

[Common Snippets]

[RPi5 snippets]

Snippet prefixes: The prefixes supported by mkqnximage are

ifs_files: Allows for additional files to be included in the ifs partition.

system_files: Allows for additional files to be included in the system partition.

data_files: Allows for additional files to be included in the data partition.

ifs_env: Used mainly for creating **procmgr symlinks** and **setting environment variables** at the beginning of the IFS start-up script.

ifs_start: Commands used for starting the system. This includes security policy loading if appropriate. It is inserted at the end of the IFS start-up script, just before execution of startup.sh or slm.

post_start: Included in **post_startup.sh a shell script run at the end of startup.sh** and by the slm configuration file.

slm: Included in the slm configuration file.

profile: Included in /system/etc/profile.

passwd_file: Include content in the passwd file.

shadow_file: Include content in the shadow file.

group_file: Include content in the group file.

uids: Add uid definitions.

boot: Used to generate the boot entry in the IFS build file.

definitions: Add content to `_definitions.inc`, the file or C preprocessor definitions used to modify the contents of other files.

options: Used to add procnto command line options and expand the size of disk partitions and free inodes.

generate: Used to generate files out of snippet files.

secpolgen: Used to configure commands used when starting secpolgenerate.

secpolused: Contains secpol rules that should be treated as having been used and thus will be added to generated policies.

QNX CTI project Details

From Snippets to SD Card Image

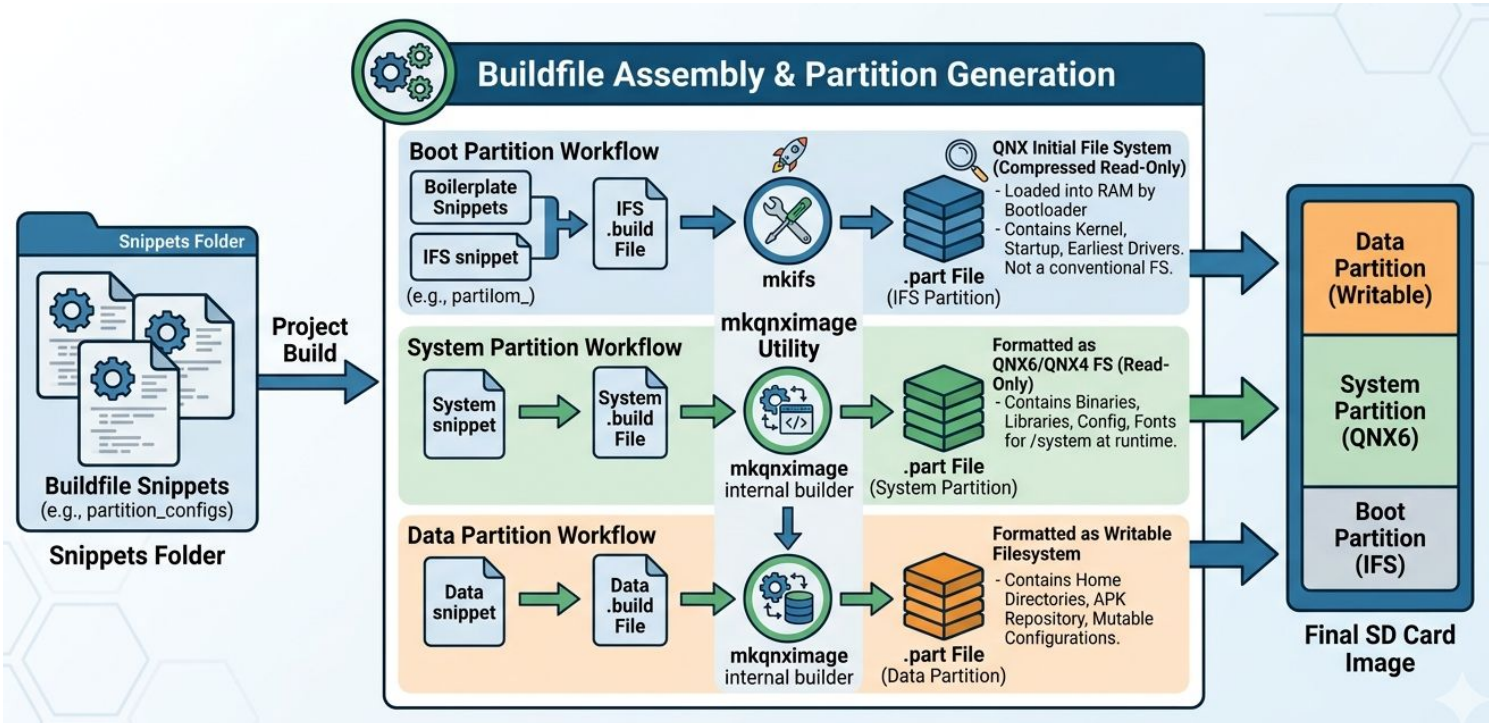
- A `.part` file is the **binary output** produced when a builder tool processes a `.build` file. It is a raw filesystem image — a stream of bytes that represents a complete, mountable partition. If you were to attach it to a loop device on Linux, you could mount it and browse its contents as a live filesystem.
- The relationship is exactly analogous to the relationship between a C source file and a compiled object file. The `.build` file is the source. The `.part` file is the compiled binary artifact.

For each of the three partitions, a different tool runs:

- For the **boot partition**, `mkifs` processes the IFS `.build` file and produces a `.part` file that contains the QNX Initial File System — the compressed in-memory filesystem that exists before any storage partition is mounted. This is special because it is not a conventional filesystem like ext4 or QNX4. It is a read-only image that the bootloader decompresses directly into RAM. The kernel, startup binary, and the earliest drivers all live here.
- For the **system partition**, a different builder (internally invoked by `mkqnximage`) processes the system `.build` file and produces a `.part` file formatted as a QNX6 or QNX4 filesystem. This contains your binaries, shared libraries, configuration files, fonts, and everything else that lives under `/system` at runtime.
- For the **data partition**, the same process produces a `.part` file formatted as a writable filesystem, containing home directories, the APK repository, and runtime-mutable configuration.

QNX CTI project Details

From Snippets to SD Card Image



QNX

QNX CTI project Details

apk/ Folder :

- The `apk` folder contains the portion of the build that integrates software packages from `oss.qnx.com`. This folder is a port of Alpine Linux's package manager (`apk`) to QNX, which allows QNX and open source software to be prepackaged as binaries.
- It is the Alpine Package Keeper, a lightweight package management system that QNX has ported to its own ecosystem.
- The purpose is to give QNX a proper on-device package manager so that software can be installed, upgraded, and removed after the image is already flashed — you are not forced to rebuild the entire image just to add a new tool.
- `world.preinstall_aarch64` and `world.preinstall_x86_64`. These files include lists of high-level apk packages representing the content to be installed in the image.
    - Artifacts are pre-installed in the `apk/stage/apk_root`.
    - And snippets are generated for the content in `apk/stage/snippets` so that they are integrated into the image.
- The critical architectural point here is that the `apk` folder serves two roles simultaneously.
    - During build time, it provides a local repository of pre-downloaded package binaries that get packed into the data partition of the SD card image.
    - At runtime on the device, the APK tool reads `/etc/apk/world` to know which packages should be present and can pull updates from the remote repository over the network. This gives you the best of both worlds — a fully offline-capable image that can also be updated live.

QNX

QNX CTI project Details

How to use Alpine Linux's package(APK) on your QNX System:

1. **Check APK configuration exists:**
   - `cat /etc/apk/repositories`
2. **World file exists**
   - `cat /etc/apk/world`
     `"What packages installed on this system"`
3. **Update/Upgrade/Install/Remove package index**
   - `apk update`
   - `apk upgrade`
   - `apk add <package_name>`
   - `apk del <package_name>`

QNX-Ports

QNX CTI project Details

assets/ Folder:

- You can customize your CTI by integrating assets, which can be shared by multiple applications that are integrated into the project. The types of assets include icons, fonts, backgrounds, and animations.

- The key design principle of the `assets/` folder is the word "**shared**." Every asset placed here is accessible to any application in the image rather than being bundled inside a single application. This matters in an embedded system where multiple apps — the demolauncher, your custom GUI app, the boot animation renderer — all need access to the same icon set or the same fonts without duplicating those files.

QNX CTI project Details

Customizing the demolauncher background image:

- Look for this portion of the **system_files.custom.window_managers_demolauncher_config** snippet:

```
[type=file uid=0 gid=0 perms=0444]
etc/images/background_p720.png=images/background_720p.png
```

- And then add a similar entry for your own image added to the folder **assets/images.**

```
[type=file uid=0 gid=0 perms=0444]
etc/images/AWADIN_Auto.png=backgrounds/local/AWADIN_Auto.png
```

- The next change required is to update the ~~**post_start.~40.window_manager**~~ **system_files.custom.apk_startup** snippet to update the demo launcher command to provide the new background image base name.
Look for this line:

```
  demolauncher -a /system/etc/desktop_files &
```

and change it to:

```
  demolauncher -a /system/etc/desktop_files -b /system/etc/images/AWADIN_Auto &
```

QNX CTI project Details

Configuring Screen

Changing the display resolution :

- To make change for this project:
  - pull/copy the contents of the file `qnx800/target/qnx/aarch64le/usr/lib/graphics/drm-rpi5/graphics-rpi5-2xhdmi.conf`
  - into the target specific snippet `system_files.custom.graphics(Create It)` to make it an inline declaration and make the updates to the inline declaration.
  - change “video-mode = 1920 x 1080 @ 60” into “video-mode = 1280 x 720 @ 60”

QNX CTI project Details

boot/ Folder :

- The `boot` folder contains files that are added to the boot partition. Its `Makefile` downloads required firmware from the files' source repos on GitHub during the build. There are two types of boot files: configuration files and firmware files.
- This folder is fundamentally different from all others in the project. It does not feed the system or data partitions at all — it feeds the FAT32 boot partition exclusively. Everything in this partition must be readable by the Raspberry Pi's VideoCore GPU firmware before any CPU code runs, which is why it must be a plain FAT32 partition with no QNX-specific filesystem structures.

Configuration files

You can modify the following configuration files directly in the boot partition either before running a build or after the image is built and flashed. The system checks the files during the next device boot: `network` and `wpa_supplicant.conf`.

- The `network` file is particularly powerful because it is read at every boot.
  The variables it controls are `HOSTNAME` to set the device hostname, `IP_ADDR` to set a static IP address for Ethernet interfaces.

-> The file is specified in each target's **boot_files.custom** snippet. You can modify its contents there on a per-target basis or edit the **/boot/network** file directly after the image has been built.

QNX CTI project Details

Configure your network settings for Wi-Fi

wpa_supplicant.conf

Ths file is used to override Wi-Fi network details prior to first boot or on subsequent boots.

```
network={
  ssid="<SSID>"
  key_mgmt=WPA-PSK
  psk="<PASSWORD>"
}
```

`wpa_supplicant.conf.verbose`. This file contains all supported options that can be set via the `wpa_supplicant.conf` file, along with comments and default values

wifi.custom (optional)

QNX CTI project Details

src/ Folder :

- The `src/` folder is the open-source compilation pipeline of the CTI project. Every project listed here is something that does not exist as a pre-built QNX binary in the SDP — it must be cross-compiled from source specifically for QNX. The `src/Makefile` manages the entire process of cloning, patching, building, and staging these projects.
- The `src/` folder is a **cross-compilation pipeline manager**. It does not store source code permanently — it clones, builds, and discards. The only permanent artifacts it produces are the compiled outputs staged into `src/stage/`, which then become available to `mkqnximage` through `OPT_REPOS`. The `src/Makefile` is the engine driving this entire pipeline, and every integration you add is expressed as a set of `make` targets inside that Makefile.
- **The three categories of source projects**
  - **QNX Ports** are projects from `github.com/qnx-ports`. These are open-source projects that the QNX community has already patched and adapted to build on QNX. They use a QNX-specific build system called `build-files` that wraps the original project's build system.
  - **QNX Projects** are projects from `gitlab.com/qnx/projects`. These are projects written by QNX engineers specifically for QNX hardware, using QNX's native `make` system directly without a third-party build system wrapper.
  - **Additional open-source projects** are projects from arbitrary upstream sources that use standard build systems like CMake or Autoconf, with patches applied where necessary to fix QNX-specific incompatibilities.

QNX

QNX CTI project Details

Steps to use the src/ folder :

Each category requires a slightly different set of commands in the build target, but the three-step structure — download target, build target, add to PKGS — is identical for all of them.

Step 1 — The download target (`source/<project>-ready`)

This target's sole responsibility is to get the source code onto your build host machine in a clean, reproducible state. The naming convention `source/<project>-ready` is mandatory — the build system looks for targets with exactly this suffix to know which projects need their source fetched.

For a QNX Ports project the target looks like this:

```
source/bash-ready:
	mkdir -p source
	cd source && git clone https://github.com/qnx-ports/bash.git
	cd source/bash && git checkout $(BASH_SHA)
	touch $@
```

- `mkdir -p source` creates the `source/` subdirectory inside `src/` if it does not already exist
- `git clone` fetches the entire repository.
- `git checkout $(BASH_SHA)` pins the working tree to an exact commit SHA defined as a variable at the top of the Makefile
- `touch $@` creates an empty file named `source/bash-ready` — this is the standard `make` trick for tracking completion of a target that does not naturally produce a single output file.

*** Pinned SHAs give you bit-for-bit reproducible builds.**

For a project that requires a patch before building:

```
source/SDL_net-ready:
	mkdir -p source
	cd source && git clone https://github.com/libsdl-org/SDL_net.git -b $(SDL_NET_VERSION)
	cd source/SDL_net && git apply $(PWD)/patches/SDL_net.patch
	touch $@
```

- The `patches/` folder inside `src/` stores these patch files.

For a local project that is developed inside the repo itself rather than fetched from the internet:

```
source/qnx-lottie_thorvg-ready:
	mkdir -p source
	cp -r $(PWD)/local/qnx-lottie_thorvg source/
	touch $@
```

QNX CTI project Details

Each category requires a slightly different set of commands in the build target, but the three-step structure — download target, build target, add to PKGS — is identical for all of them.

Step 2 — The build target (`source/<project>-built-$(QNX_ARCH)`)

This target cross-compiles the project for the target architecture. The `$(QNX_ARCH)` suffix in the filename is what makes the build system architecture-aware — when you run `make TARGET=rpi5`, `QNX_ARCH` resolves to `aarch64le`, so the target becomes `source/bash-built-aarch64le`. If you later run `make TARGET=qemu`, the target becomes `source/bash-built-x86_64`, and `make` correctly identifies them as different targets requiring separate builds.

For a QNX Ports:

```
source/bash-built-$(QNX_ARCH): source/bash-ready source/build-files-ready
        QCONF_OVERRIDE=$(PWD)/qconf-override.mk
        CPULIST=$(QNX_ARCH) \
        QNX_ARCH=$(QNX_ARCH) \
QNX_PROJECT_ROOT="$(PWD)/source/bash" make -C
source/build-files/ports/bash install -j4
touch $@
```

- `source/bash-ready` and `source/build-files-ready` are declared as dependencies “`make` will not execute this target until both those targets have completed successfully”
- `QCONF_OVERRIDE` points to a file that overrides QNX build configuration to target your specific arch, and `CPULIST` restricts the build to the architecture you are currently building for
- `make -C source/build-files/ports/bash install` runs the QNX Ports build system for bash, placing the compiled binaries and headers into the stage directory. The `install` target in QNX build-files always copies outputs to stage.”`src/stage/nto/aarch64le/` for target-specific outputs and `src/stage/nto/` for architecture-neutral”

For a QNX Projects project (native QNX make)

```
source/rpi-gpio-built-$(QNX_ARCH): source/cairo-built-$(QNX_ARCH)
source/rpi-gpio-ready
cd source/rpi-gpio && \
QCONF_OVERRIDE=$(PWD)/qconf-override.mk \
QNX_ARCH=$(QNX_ARCH) \   make hinstall
cd source/rpi-gpio && \
EXTRA_INCVPATH=$(STAGE_COMMON)/usr/local/include \
EXTRA_LIBVPATH=$(STAGE_TARGET)/usr/local/lib \
MY_STAGE=$(STAGE_ROOT) make    cd source/rpi-gpio && \
QCONF_OVERRIDE=$(PWD)/qconf-override.mk \
QNX_ARCH=$(QNX_ARCH) \
EXTRA_INCVPATH=$(STAGE_COMMON)/usr/local/include \
EXTRA_LIBVPATH=$(STAGE_TARGET)/usr/local/lib \
MY_STAGE=$(STAGE_ROOT) make install      touch $@
```

- `make hinstall` installs the public headers into the stage first — this is necessary because `rpi-thermal`
- `EXTRA_INCVPATH` and `EXTRA_LIBVPATH` variables tell the QNX compiler where to find headers and libraries from other projects that were staged earlier

QNX CTI project Details

Step 2 — The build target (`source/<project>-built-$(QNX_ARCH)`)

For a CMake-based open-source project

```
source/SDL_net-built-$(QNX_ARCH): source/SDL_net-ready source/SDL-built-$(QNX_ARCH)
	mkdir -p source/SDL_net/build-$(QNX_ARCH)
	cd source/SDL_net/build-$(QNX_ARCH) && cmake .. \
		-DCMAKE_TOOLCHAIN_FILE=$(PWD)/patches/$(QNX_ARCH)-qnx.cmake \
		-DCMAKE_BUILD_TYPE=Release \
		-DCMAKE_INSTALL_PREFIX=$(STAGE_TARGET) \
		-DSDL2_LIBRARY=$(STAGE_TARGET)/lib/libSDL2-2.0.so \
		-DSDL2_INCLUDE_DIR=$(STAGE_TARGET)/include/SDL2
	cd source/SDL_net/build-$(QNX_ARCH) && make
	cd source/SDL_net/build-$(QNX_ARCH) && make install
	touch $@
```

- `CMAKE_TOOLCHAIN_FILE`. This points to a CMake toolchain file stored in `src/patches/` that tells CMake to use the QNX cross-compiler instead of the host GCC, sets the correct sysroot, and configures the correct target ABI. Without this file, CMake would compile for your host machine instead of for AArch64 QNX.

QNX CTI project Details

Step 3 — Adding to the PKGS variable

```
PKGS = bash vim cairo simple-terminal screenwm SDL SDL_image SDL_ttf SDL_net pattern-race
```

- The `PKGS` variable is a space-separated list of all project names to build. The top-level `src/Makefile` iterates over this list and ensures both the `source/<project>-ready` and `source/<project>-built-$(QNX_ARCH)` targets are executed for each entry.

- For platform-specific projects:

```
ifneq ($(filter rpi4 rpi5,$(TARGET)),)
PKGS += rpi-gpio rpi-mailbox rpi-thermal qnx-lottie_thorvg
endif
```

Step 4 — Making the binary visible to the image builder

Tell `mkqnximage` to include its output in the partition. This is done in a `system_files` snippet

```
[type=file uid=0 gid=0 perms=0755] bin/my-app=usr/local/bin/my-app
```

QNX CTI project Details

Example : Adding your own project — the complete recipe

First, add version control variables at the top of `src/Makefile`:

```
MY_SENSOR_SHA = a3f8c21d9b04e7f5…
```

Second, write the download target:

```
source/my-sensor-driver-ready:
	mkdir -p source
	cd source && git clone https://gitlab.com/your-org/my-sensor-driver.git
	cd source/my-sensor-driver && git checkout $(MY_SENSOR_SHA)
	touch $@
```

Third, write the build target (assuming it uses QNX native make):

```
source/my-sensor-driver-built-$(QNX_ARCH): source/my-sensor-driver-ready
	cd source/my-sensor-driver && \
	QCONF_OVERRIDE=$(PWD)/qconf-override.mk \
	QNX_ARCH=$(QNX_ARCH) \
	MY_STAGE=$(STAGE_ROOT) make install
	touch $@
```

Fourth, add to PKGS, guarded for RPi5 only:

makefile

```
ifneq ($(filter rpi5,$(TARGET)),)
PKGS += my-sensor-driver
endif
```

Fifth, create `snippets/system_files.custom.my_sensor_driver`:

```
[type=file uid=0 gid=0 perms=0755]
bin/my-sensor-driver=usr/local/bin/my-sensor-driver
```

Sixth, run `make TARGET=rpi5`.

The Makefile will clone your repo, cross-compile it, stage the output, and `mkqnximage` will pack it into the system partition automatically. The entire chain from source to flashed binary is fully automated and reproducible.

QNX

QNX CTI project Details

Tasks :

1- Clone your Code and add it to the project

2- Add Pattern Race : is a simple pattern-matching game that uses three of the SDL libraries.

3- **screenwm:** is an alternative window manager that you can use in place of the default fullscreen window manager.

QNX CTI project Details

system/ Folder :

- You can use the `system` folder for files that you want to add directly to the **system partition**.
- This is in contrast to **binaries and configuration** files that are part of QNX Software Center packages, as well as **open-source projects that are downloaded and built**.
- Aside from a couple of exceptions, the relative path in the folder is usually consistent with the relative file intended for the file to be placed in the system partition, but this mapping is controlled by **the snippet that includes each file.**
- If `src/` is for things that must be built, `system/` is for things that are already built and just need to be placed into the image.
- The following configuration categories and their locations are:
  - SPI configuration at `etc/config/spi`,
  - camera and touchscreen configuration at `etc/system/config`,
  - RPi5-specific at `etc/system/config/rpi5`,
  - miscellaneous support scripts for the Developer Desktop at `usr/bin`,
  - desktop modifications for user `.config` at `desktop.config`,
  - and desktop modifications for user `.local` at `desktop.local`.

QNX CTI project Details

Touchscreen on RPI5 :

- If you connect a touchscreen using a micro HDMI to HDMI and USB cable to your Raspberry Pi, the touch function should work by default.
- If the driver fails to detect the touchscreen, you can manually set up the **mtouch.conf** configuration file:

1. Ensure that `io-hid -d usb` is running. To do so, run **`pidin ar`** to view the active processes.
2. Run **`hidview`** to get the list of USB devices. Find your touchscreen and record its Vendor and Product ID.
3. Set up the `mtouch "section specifies the configuration to apply touch devices (also called mtouch devices for multitouch devices)"` configuration file."Touch configuration file"

```
begin mtouch
 driver = hid
 options = vid=0x0eef,did=0x7200,verbose=6,width=4096,height=4096,max_touchpoints=1
 end mtouch
```

QNX

QNX CTI project Details

targets/ Folder :

The `targets/` folder is the board abstraction layer of the entire CTI project. Everything that differs between one hardware platform and another lives here, isolated from the shared project content that is common to all platforms.

- The `qsc_install_packages.list` file means your target requires additional or different packages to be installed into the local or separate installation of the SDP.
- The `variables.mk` file is required because the build process requires variables to be set by each target. You must set `ALL_TARGET` which is the name of the final `make` target generated, usually the final image that the build process produces. The contents of this variable are added to the `all` makefile target automatically. `QNX_ARCH` is the target's architecture, typically `aarch64` or `x86_64`. `QNX_ARCHDIR` is the name of the directory that holds architecture-specific files in the SDP and the `src` tree's stage.
- The `mkqnximage.config` file is the configuration file passed to `mkqnximage`. This is the file you have already studied in detail — partition sizes, `OPT_TYPE`, `OPT_REPOS`, and all the feature flags.

Thank You

Goto the Kahoot quiz